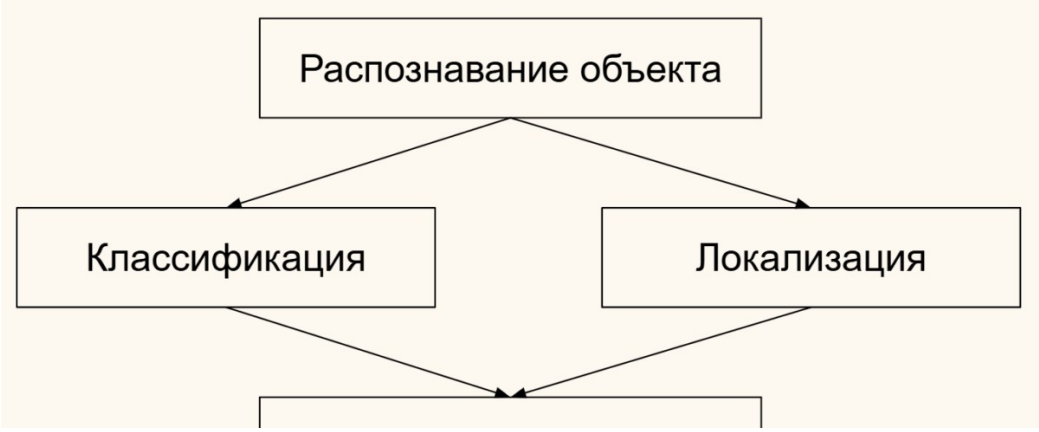
Свёрточная нейронная сеть. Часть 1

Технологии машинного обучения сделали большой шаг вперед в работе с цифровыми изображениями. В частности, сегодня сложно удивить возможностями по классификации изображений (image classification) или обнаружению объектов (object detection). Более интересным вопросом является то, *как именно* модели ML показывают такую высокую точность при решении задач компьютерного зрения.

Ключевым инструментом в этой области является так называемая **свёрточная нейронная сеть** (convolutional neural network, CNN, а также ConvNet).

Задачи компьютерного зрения

Вначале опишем основные задачи **компьютерного зрения** (computer vision). Так мы будем лучше понимать, для каких целей можно использовать свёрточную нейронную сеть.



Наиболее общей задачей является **распознавание объекта** (object recognition), то есть понимание или анализ компьютером того, что представлено на цифровом изображении или на видео.

Внутри этой общей задачи стоят подзадачи **классификации** (object classification), то есть отнесения объекта к определенному классу, и **локализации** (object localization) или указания (с помощью рамки) положения объекта на изображении.

Обнаружение объекта (object detection) предполагает как классификацию, так и локализацию объекта. В то же время на этом этапе мы не можем ничего сказать о площади или, например, периметре объекта. Для решения этой задачи необходимо выполнить **сегментацию изображения** (image segmentation), то есть разделение изображения на отдельные области с выделенными границами.

Добавим, что задачи компьютерного зрения решаются не только алгоритмами глубокого обучения, но и с помощью классического ML. В качестве примера можно привести **метод Виолы-Джонса** (Viola-Jones method) для обнаружения лиц.

Полносвязные нейросети

Посмотрим, почему не стоит применять «обычные» полносвязные нейросети для работы с изображениями.

Переобучение. Вспомним, что когда мы предсказывали рукописные цифры из датасета MNIST, архитектура применяемой нами

полносвязной нейронной сети состояла из входного, двух скрытых и одного выходного слоя.

Каждое изображение в этом наборе данных имело размерность 28 на 28 пикселей, что при «вытягивании» давало 784 нейрона входного слоя. Каждый скрытый слой состоял из 64 нейронов и наконец выходной слой насчитывал 10 нейронов по числу цифр или классов. Всего даже при использовании таких небольших изображений получалось 55 050 параметров.

Помимо вычислительной затратности алгоритма такое количество нейронов и параметров модели способствует её переобучению.

Сохранение структуры изображения. Еще один повод отказаться от полносвязной архитектуры заключается в том, что в отличие от, например, задачи предсказания цены квартиры по ее параметрам, где, скажем район, скорее всего, не коррелирует с количеством комнат, в изображениях расположенные рядом пиксели (вспомним, что пиксели являются признаками) связаны между собой.

В самом деле, если на картинке изображена рыжая кошка, то вероятность того, что рядом с «рыжим» пикселем будет пиксель такого же оттенка выше, чем вероятность пикселя другого цвета. Таким образом, «вытягивая» матрицу картинки, мы уничтожаем то, что называется **пространственной структурой** (spatial structure) изображения.

Пространственная инвариантность. Добавим, что поскольку полносвязная сеть запоминает каждый пиксель отдельно, то она будет ожидать, что кошка всегда расположена в определенной части изображения. При этом мы понимаем, что в реальности объект может находиться в любой части картинки. Говорят, что полносвязная нейросеть не обладает **пространственной инвариантностью** (spatial invariance).

Именно эти недостатки полносвязной архитектуры и призвана

преодолеть свёрточная нейронная сеть.

Дискретная свёртка

Операция свёртки

Начнем с не связанного с компьютерным зрением и нейросетями примера. Предположим, что вы работаете в больнице, и к вам на амбулаторное лечение приходят пациенты, каждому из которых нужно дать по три таблетки лекарства.

Предположим, что в течение пяти дней к вам приходит 1, 2, 3, 4 и 5 пациентов.

Дни	Пн	Вт	Ср	Чт	Пт
Кол-во пациентов	1	2	3	4	5

Поставим задачу рассчитать, какое количество препарата потребуется в каждый из приемных дней. Вполне очевидно, что достаточно умножить (обозначим такое умножение через $*$) три таблетки на количество пациентов в каждый из дней лечения.

$$3 * [1, 2, 3, 4, 5] = [3, 6, 9, 12, 15]$$

Теперь допустим, что план лечения изменился и каждому пациенту необходимо в первый день дать три таблетки, во второй — две, а в третий — одну.

$$[3, 2, 1] * [1, 2, 3, 4, 5]$$

Другими словами, в понедельник придет один пациент и получит три таблетки, во вторник придут еще два пациента и тоже получат три таблетки, а первый пациент получит уже две таблетки. Таким образом,

в первый день потребуется $1 \cdot 3 = 3$ единицы препарата, а во второй $1 \cdot 2 + 2 \cdot 3 = 8$ единиц препарата. Очевидно, что с течением времени вести учет потребности в лекарстве становится довольно сложно.

Попробуем упростить расчеты следующим образом. Разместим элементы списка пациентов в обратном порядке, т.е. [5, 4, 3, 2, 1], а список с количеством таблеток разместим над ним таким образом, чтобы, например, в первый день пришедший пациент соответствовал трем необходимым ему таблеткам.

```
          [ 3, 2, 1 ]
[ 5, 4, 3, 2, 1 ]
```

Перемножим количество пациентов и количество таблеток и получим потребность в лекарстве в понеделник, $1 \cdot 3 = 3$. Далее сдвинем список пациентов на одно значение вправо.

```
          [ 3, 2, 1 ]
  [ 5, 4, 3, 2, 1 ] →
```

Просуммируем те значения обоих списков, которые находятся друг над другом и сложим результат, $1 \cdot 2 + 2 \cdot 3 = 8$. Продолжая сдвигать таким же образом список пациентов, мы получим, что потребность в препарате составляет

Дни	Пн	Вт	Ср	Чт	Пт	Сб	Вс
Кол-во препарата	3	8	14	20	26	14	5

В субботу и в воскресенье потребность в препарате возникает из-за того, что пришедшие в пятницу пациенты будут проходить трехдневный курс лечения.

Описанную выше операцию назовем (дискретной) **свёрткой** или

конволюцией (convolution) и обозначим через оператор $*$. Таким образом,

[3, 2, 1] $*$ [1, 2, 3, 4, 5] = [3, 8, 14, 20, 26, 14, 5]

Математически операцию свёртки можно задать следующим образом. Введем функцию $f(x)$, определяющую количество лекарства, а также функцию $g(-x+t)$, определяющую количество пациентов в каждый из дней лечения.

Визуально аргумент $g(-x+t)$ подразумевает, что мы отражаем g относительно оси ординат $-x$ и сдвигаем на $+t$ вправо в зависимости от того, для какого дня мы хотим рассчитать количество препарата.

Общая формула будет иметь вид

$$(f*g)[t] = \sum_{x=-\infty}^{\infty} f(x) \cdot g(-x+t)$$

Заметим, что чаще вместо $g(-x+t)$ пишут $g(t-x)$.

Границы суммирования идут от $-\infty$ до ∞, чтобы учесть все возможные значения аргумента. Для функций с конечным носителем, т.е. конечным подмножеством области определения, на котором функция отлична от нуля (как в нашем случае) мощность носителя свёртки определяется по формуле

$$N+M-1,$$

где N — мощность носителя функции f, а M — мощность носителя g. Таким образом, в нашем примере $N=3, M=5$ и мощность носителя свёртки будет равна $3+5-1=7$.

Откроем блокнот к занятию⧉

В Питоне свёртку одномерных массивов можно выполнить с помощью

функции **np.convolve()**.

```
drugs, patients = np.array([3,2,1]), np.array([1,2,3,4,5])
np.convolve(drugs, patients)
```

```
array([ 3,  8, 14, 20, 26, 14,  5])
```

Коммутативность свёртки

Несложно увидеть, что операция свертки коммутативна. Если мы зафиксируем список пациентов и перевернем и будем сдвигать план лечения, то результат не изменится.

$$\begin{array}{r} [\,1, 2, 3, 4, 5\,] \\ [\,1, 2, 3\,] \rightarrow \end{array}$$

```
np.convolve(patients, drugs)
```

```
array([ 3,  8, 14, 20, 26, 14,  5])
```

Таким образом,

$$f * g = g * f$$

Свёртка двумерных массивов

Перейдем к двумерным массивам и сразу начнем говорить про изображения. Ведь, как мы помним, цифровое черно-белое изображение представляет собой матрицу (двумерный массив), в которой значения каждого элемента указывают на интенсивность оттенка серого. Оттенки находятся в диапазоне от 0 (черный) до 255 (белый).

Рассмотрим следующее небольшое изображение размером 6 x 6 пикселей.

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Кроме этого, возьмем еще одну матрицу меньшей размерности.

1	0	−1
1	0	−1
1	0	−1

Свёртку этих массивов будем производить следующим образом. На **итерации 1** поместим меньшее изображение поверх большего в левый верхний угол последнего. Значения меньшей матрицы укажем через надстрочные индексы.

$3^{(1)}$	$0^{(0)}$	$1^{(-1)}$	2	7	4
$1^{(1)}$	$5^{(0)}$	$8^{(-1)}$	9	3	1
$2^{(1)}$	$7^{(0)}$	$2^{(-1)}$	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Перемножим совпадающие элементы и сложим произведения.

$$3 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 0 \cdot 0 + 5 \cdot 0 + 7 \cdot 0 +$$
$$1 \cdot (-1) + 8 \cdot (-1) + 2 \cdot (-1) = -5$$

Полученное значение поместим в матрицу размерностью 4 x 4 пикселя.

−5			

На **итерации 2** сместим меньшую матрицу на одну клетку вправо и проведем аналогичные расчеты.

3	$0^{(1)}$	$1^{(0)}$	$2^{(-1)}$	7	4
1	$5^{(1)}$	$8^{(0)}$	$9^{(-1)}$	3	1
2	$7^{(1)}$	$2^{(0)}$	$5^{(-1)}$	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

$$0 \cdot 1 + 5 \cdot 1 + 7 \cdot 1 + 1 \cdot 0 + 8 \cdot 0 + 2 \cdot 0 +$$
$$2 \cdot (-1) + 9 \cdot (-1) + 5 \cdot (-1) = -4$$

−5	−4		

Продолжая перемещать меньшую матрицу по большей, мы заполним всю матрицу 4 x 4.

−5	−4	0	8
−10	−2	2	3
0	−2	−4	−7
−3	−2	−3	−16

Это и будет результатом свёртки двух двумерных массивов.

Свёртка и взаимная корреляция

Попробуем получить такой же результат с помощью Питона. Создадим соответствующие массивы.

```
big = np.array([[ 3, 0, 1, 2, 7, 4 ],
                [ 1, 5, 8, 9, 3, 1 ],
                [ 2, 7, 2, 5, 1, 3 ],
                [ 0, 1, 3, 1, 7, 8 ],
                [ 4, 2, 1, 6, 2, 8 ],
                [ 2, 4, 5, 2, 3, 9 ]])

small = np.array([[ 1, 0, -1 ],
                  [ 1, 0, -1 ],
                  [ 1, 0, -1 ]])
```

Применим функцию **convolve2d()** из модуля signal библиотеки scipy.

```
from scipy import signal
signal.convolve2d(big, small, mode = 'valid')
```

```
array([[ 5,  4,  0, -8],
       [10,  2, -2, -3],
       [ 0,  2,  4,  7],
       [ 3,  2,  3, 16]])
```

Как мы видим, результат отличается от того, который мы получили выше. Дело в том, что мы забыли «перевернуть» либо больший, либо меньший массив и та операция, которую мы выполнили называется **взаимной корреляцией** (cross-correlation) и обозначается через оператор $\star$.

Применим функцию **correlate2d()**.

```
signal.correlate2d(big, small, mode = 'valid')
```

```
array([[ -5,  -4,   0,   8],
       [-10,  -2,   2,   3],
       [  0,  -2,  -4,  -7],
       [ -3,  -2,  -3, -16]])
```

Убедимся, что результат функции **convolve2d()** можно получить, разместив элементы меньшей матрицы в обратном порядке (причем теперь уже как для строк, так и для столбцов, поскольку речь идет о двумерных объектах) и затем применив функцию **correlate2d()**.

Для наглядности покажем, что значит разместить элементы матрицы в обратном порядке в двух измерениях.

a	b	c
d	e	f
g	h	i

i	h	g
f	e	d
c	b	a

Применим подобное преобразование к меньшей матрице, а затем выполним взаимную корреляцию.

```
# функция np.flipud() от англ. up-down переворачивает строки,
# а функция np.fliplr() от англ. left-right соответственно столбцы
small_flipped = np.flipud(np.fliplr(small))
small_flipped
```

```
array([[-1,  0,  1],
       [-1,  0,  1],
       [-1,  0,  1]])
```

```
signal.correlate2d(big, small_flipped, mode = 'valid')
```

```
array([[ 5,  4,  0, -8],
       [10,  2, -2, -3],
       [ 0,  2,  4,  7],
       [ 3,  2,  3, 16]])
```

Сравните с результатом свёртки выше. Математически, операции взаимной корреляции и свёртки двух матриц F (большая матрица) и g (малая) можно записать так

$$H_{corr} = F \star g, \quad H_{corr}[i,j] \sum_{x_1=-\infty}^{\infty} \sum_{x_2=-\infty}^{\infty} F[x_1, x_2] g[i + x_1, j + x_2]$$

$$H_{conv} = F * g, \quad H_{conv}[i,j] \sum_{x_1=-\infty}^{\infty} \sum_{x_2=-\infty}^{\infty} F[x_1, x_2] g[i - x_1, j - x_2]$$

По сравнению с одномерной свёрткой здесь мы суммируем по двум измерениям, смещая g на i шагов по горизонтали и на j шагов по вертикали. Ключевым отличием в этих формулах будет замена $[i + x_1, j + x_2]$ на $[i - x1, j - x2]$, что и будет во втором случае отвечать за размещение элементов в обратном порядке.

Забегая вперед скажем, что в свёрточных нейронных сетях в действительности используется операция взаимной корреляции, а не свёртки, поскольку элементы матриц не размещаются в обратном порядке.

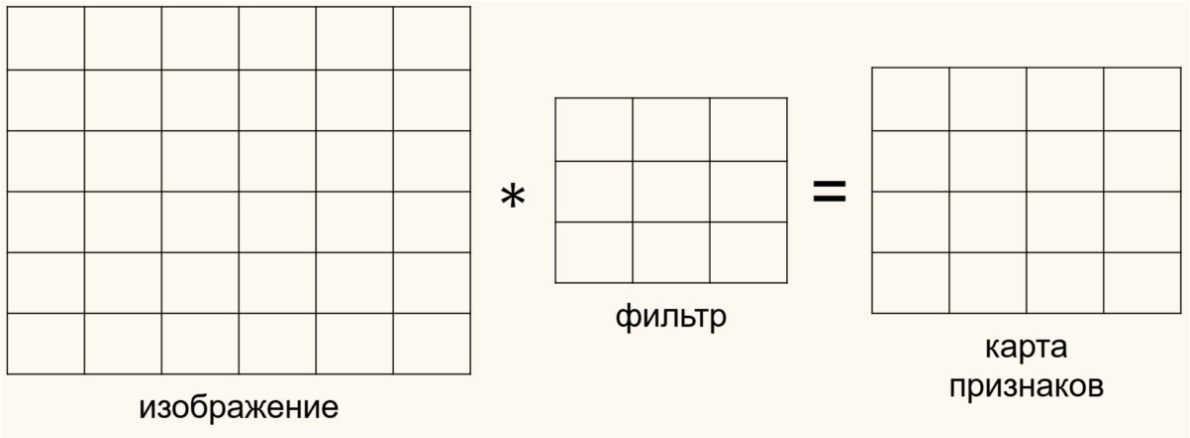
И хотя свёртка и взаимная корреляция — разные операции, дающие разный результат, в случае свёрточных нейронных сетей отсутствие «переворачивания» матриц, как мы увидим позднее, никак не влияет

на их способность запоминать закономерности на изображениях. В других приложениях, в частности, при расчете потребности в препаратах, эти операции нельзя назвать взаимозаменяемыми.

Одновременно в дальнейшем, следуя общепринятой терминологии, мы будем говорить именно об операции свёртки.

Немного терминологии

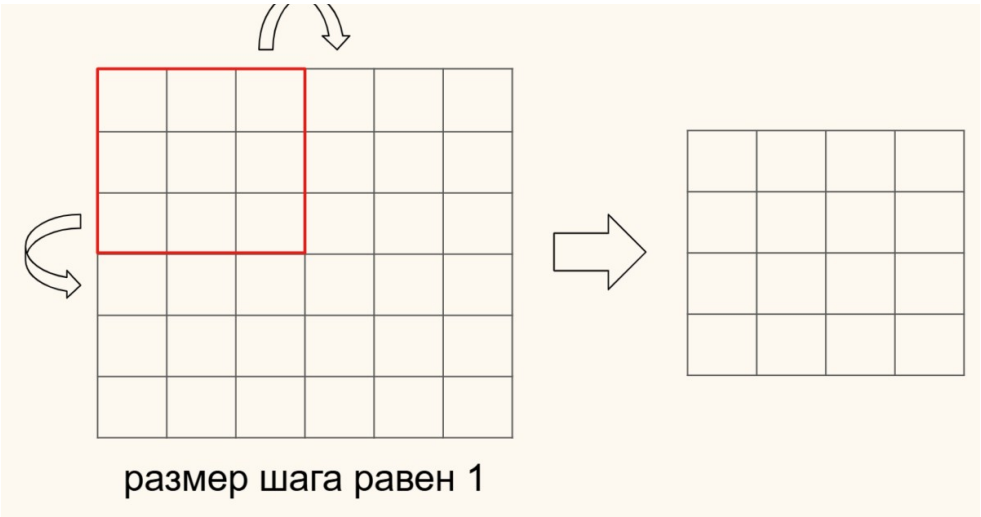
Большая матрица, как мы уже сказали, является исходным **изображением** (image), к которому мы применяем малую матрицу, называемую **фильтром** (filter). Результатом свёртки будет еще одна матрица, называемая **картой признаков** (feature map).



Изображение также называют **сигналом** (signal), а фильтр — **ядром** (kernel).

Про карту признаков

На рисунке выше размерность карты признаков составляет 4 x 4, потому что при условии, что мы (1) на каждом шаге сдвигаем фильтр на одну клетку и (2) не выходим за границы изображения, мы можем заполнить не более четырех столбцов и не более четырех строк.



Для лучшего понимания поясним, что шагов, например, вправо мы можем сделать только три, однако не стоит забывать, что при исходном положении фильтра в левом верхнем углу мы также рассчитываем свёртку.

В общем случае, если обозначить размерность изображения через $n \times n$, а размерность фильтра через $k \times k$, размерность карты признаков можно найти по формуле

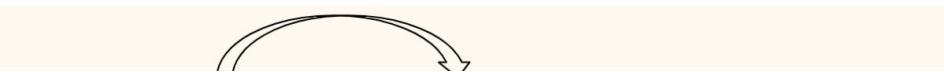
$$(n \times n) * (k \times k) \to (n - k + 1) \times (n - k + 1)$$

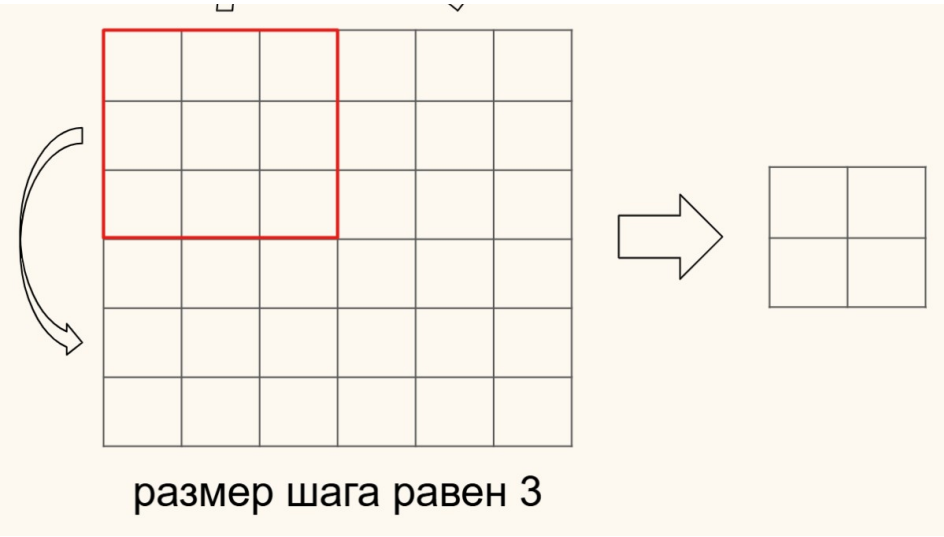
В самом деле, $(6 - 3 + 1) \times (6 - 3 + 1) \to (4 \times 4)$.

Размер шага

Фильтр можно перемещать по изображению не на одну клетку, а, например, на две. Такой параметр более точно называется **шагом по индексу** (stride) и очевидно, чем он больше, тем меньшего размера будет карта признаков.

Например, если размер шага равен трем, то тогда можно сделать лишь один шаг вправо, и столько же вниз. Таким образом, размерность карты признаков в этом случае составит 2 x 2.





Другими словами, мы в три раза уменьшили возможность смещения по горизонтали и во столько же раз по вертикали.

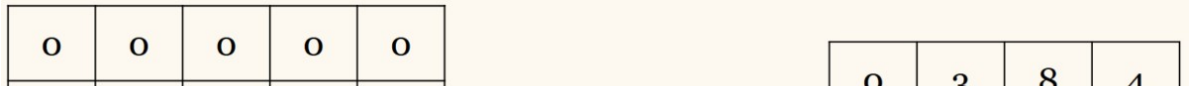
Отступ

В некоторых случаях нам не хотелось бы сокращать размер карты признаков. При этом из примеров выше очевидно, что свёртка приводит именно к такому результату.

Преодолеть эту особенность можно с помощью **отступа** (padding). Вначале рассмотрим свёртку изображения 3 x 3 с фильтром 2 x 2 размером шага 1.

$$\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} * \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$$

Ожидаемо, карта признаков уменьшилась до размерности 2 x 2. Теперь добавим «рамку» вокруг исходного изображения и заполним ее нулями, а затем выполним свёртку с тем же фильтром.



0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

*

0	1
2	3

=

9	19	25	10
21	37	43	16
6	7	8	0

Исходное изображение, таким образом, увеличится до размера 5 x 5, а карта признаков до 4 x 4.

Заметим, что `mode = 'valid'` функции **correlate2d()**, как раз запрещает использовать отступ.

```
Image = np.array([[ 0, 1, 2 ],
                  [ 3, 4, 5 ],
                  [ 6, 7, 8 ]])

filter = np.array([[0,1],
                   [2,3]])
```

```
signal.correlate2d(Image, filter, mode = 'valid')
```

```
array([[19, 25],
       [37, 43]])
```

Параметр `mode = 'full'` наоборот, предписывает отступ.

```
signal.correlate2d(Image, filter, mode = 'full')
```

```
array([[ 0,  3,  8,  4],
       [ 9, 19, 25, 10],
       [21, 37, 43, 16],
       [ 6,  7,  8,  0]])
```

Теперь давайте посмотрим, каким образом операция свёртки может применяться для обработки изображений.

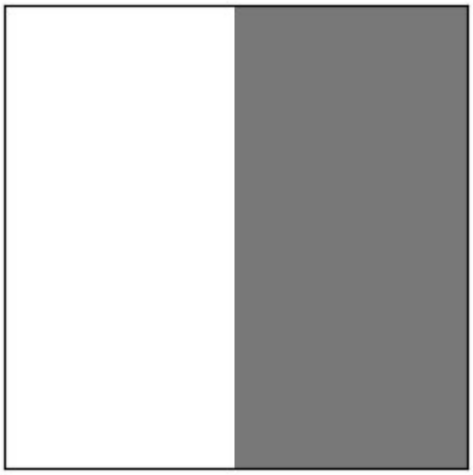
Обнаружение контуров

Простой пример

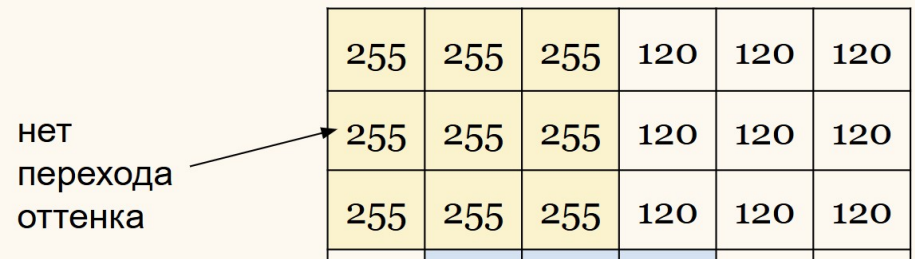
Одной из задач обработки изображений является **обнаружение контуров** (edge detection), т.е. нахождение тех мест на изображении, где интенсивность цвета резко меняется. Рассмотрим следующее изображение.

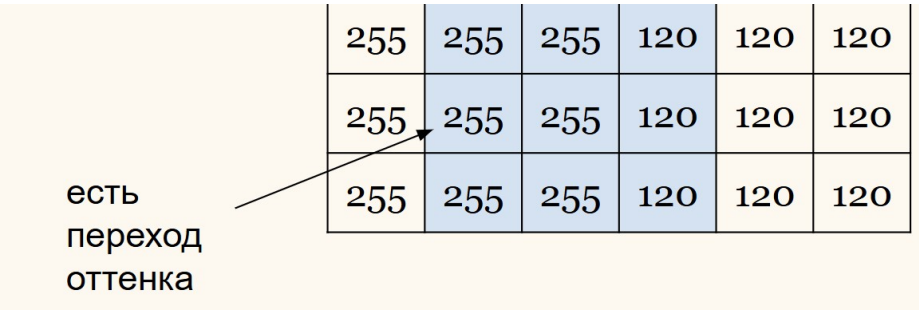
```
vertical_edge = np.array([[255, 255, 255, 120, 120, 120],
                          [255, 255, 255, 120, 120, 120],
                          [255, 255, 255, 120, 120, 120],
                          [255, 255, 255, 120, 120, 120],
                          [255, 255, 255, 120, 120, 120],
                          [255, 255, 255, 120, 120, 120]])

plt.figure(figsize = (3,3))
plt.xticks([])
plt.yticks([])
plt.imshow(vertical_edge, cmap = 'gray', vmin = 0, vmax = 255);
```



По центру изображения находится **вертикальный контур** (vertical image). Поставим задачу построить новое изображение, на котором светлым оттенком (т.е. высокими значениями пикселя) будет выделена область, в которой присутствует резкий переход интенсивности оттенков, и темным оттенком — области, в которых такого перехода нет.





Интересно, что для решения этой задачи можно использовать свёртку. Возьмем новый фильтр и применим его к изображению.

```
vertical_edge_filter = np.array([[ 1, 0, -1 ],
                                 [ 2, 0, -2 ],
                                 [ 1, 0, -1 ]])
```

Наложим фильтр на область без перехода оттенка изображения и выполним расчеты.

$$255 \cdot 1 + 255 \cdot 2 + 255 \cdot 1 + 255 \cdot 0 + 255 \cdot 0 + 255 \cdot 0 +$$

$$255 \cdot (-1) + 255 \cdot (-2) + 255 \cdot (-1) =$$

$$255 + 510 + 255 + 0 + 0 + 0 - 255 - 510 - 255 = 0$$

Как мы видим, значения фильтра подобраны таким образом, что при отсутствии контура (перехода) значения взаимоуничтожаются. Теперь выполним аналогичные расчеты для области с переходом.

$$255 \cdot 1 + 255 \cdot 2 + 255 \cdot 1 + 255 \cdot 0 + 255 \cdot 0 + 255 \cdot 0 +$$

$$120 \cdot (-1) + 120 \cdot (-2) + 120 \cdot (-1) =$$

$$255 + 510 + 255 + 0 + 0 + 0 - 120 - 240 - 120 = 540$$

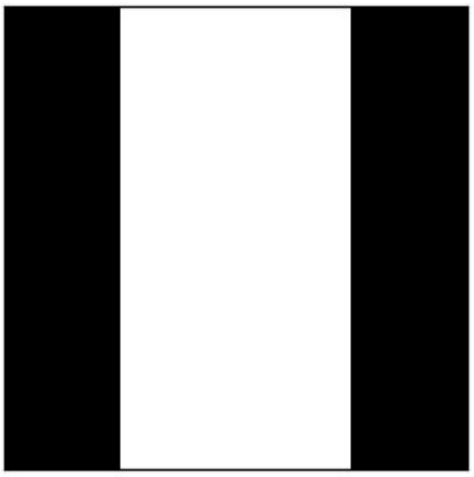
Здесь контур обнаружен и получившееся значение больше нуля. Выполним свертку изображения и фильтра.

```
feature_map = signal.correlate2d(vertical_edge, vertical_edge_filt
feature_map
```

```
array([[  0, 540, 540,   0],
       [  0, 540, 540,   0],
       [  0, 540, 540,   0],
```

```
        [  0, 540, 540,   0]])
```

```
plt.figure(figsize = (3,3))
plt.xticks([])
plt.yticks([])
plt.imshow(feature_map, cmap = 'gray', vmin = 0, vmax = 255);
```



На карте признаков хорошо видно, где на исходном изображении находится вертикальный контур.

Интересно, что приведенный выше фильтр ищет переход от светлого оттенка к темному. Если бы на изображении темный оттенок наоборот переходил в светлый, то результатом применения такого фильтра были бы отрицательные значения и, как следствие, выделение контура темным цветом.

```
# перевернем картинку относительно оси y
vertical_edge_inv = np.fliplr(vertical_edge)

plt.figure(figsize = (3,3))
plt.xticks([])
plt.yticks([])
plt.imshow(vertical_edge_inv, cmap = 'gray', vmin = 0, vmax = 255)
```

Выполним свёртку.

```
feature_map_inv = signal.correlate2d(vertical_edge_inv, vertical_e
feature_map_inv
```

```
array([[   0, -540, -540,    0],
       [   0, -540, -540,    0],
       [   0, -540, -540,    0],
       [   0, -540, -540,    0]])
```

Посмотрим на изображение.

```
plt.figure(figsize = (3,3))
plt.xticks([])
plt.yticks([])
plt.imshow(feature_map_inv, cmap = 'gray', vmin = 0, vmax = 255);
```

Для того чтобы на карте признаков получить видимый контур, нужно перевернуть фильтр.

```
vertical_edge_filter_inv = np.fliplr(vertical_edge_filter)
vertical_edge_filter_inv
```

```
array([[-1,  0,  1],
       [-2,  0,  2],
       [-1,  0,  1]])
```

```
signal.correlate2d(vertical_edge_inv, vertical_edge_filter_inv, mo
```

```
array([[  0, 540, 540,   0],
       [  0, 540, 540,   0],
       [  0, 540, 540,   0],
       [  0, 540, 540,   0]])
```

На практике используют только один вертикальный фильтр (любой из двух), а затем берут модуль получившихся значений или возводят значения в квадрат.

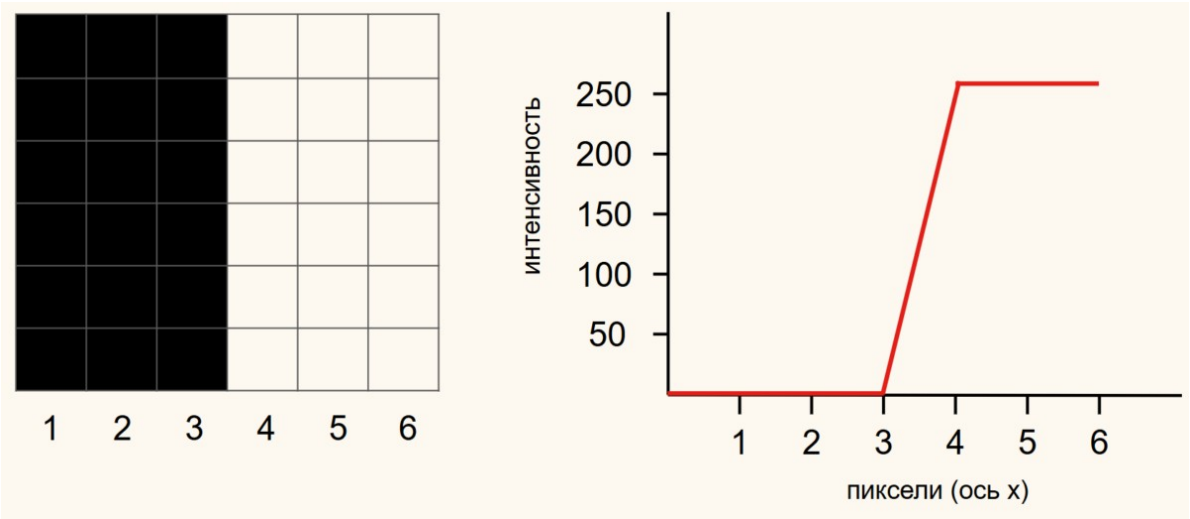
Аналогичным образом можно найти и горизонтальный контур. Для этого потребуется другой фильтр.

```
horizontal_edge_filter = np.array([[ 1, 2, 1 ],
                                   [ 0, 0, 0 ],
                                   [ -1, -2, -1 ]])
```

Фильтр Собеля

Вместе эти фильтры называются **фильтрами Собеля** (Sobel filter). Их задача заключается в том, чтобы найти *градиент* (степень изменения) как по горизонтали, т.е. по оси x, так и по вертикали, то есть по оси y.

В самом деле, переход, например, из черного в белое по оси x можно представить следующим образом.



Таким образом, математически, речь идет о нахождении двух частных производных по каждой из осей в определенной точке (для определенного пикселя).

Рассмотрим сегмент исходного изображения размерностью 3 x 3 со значением интенсивности каждого пикселя z_i.

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

Оценим градиент пикселя z_5. Зададим частные производные по обеим осям.

$$g_x = \frac{\partial_f}{\partial_x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

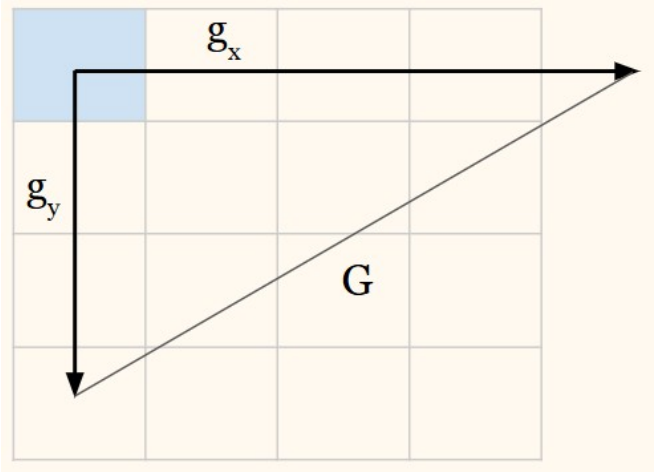
Другими словами, мы оцениваем разность интенсивности пикселей, придавая больший вес изменению второго столбца и второй строки (отсюда двойка в формуле).

Если g_x или g_y не равны нулю — это контур, в противном случае — не контур.

Поскольку, применив один из вертикальных фильтров и один из горизонтальных фильтров, мы можем получить как положительные, так и отрицательные значения частных производных, для оценки градиента конкретного пикселя удобно найти его длину (magnitude).

$$G = \sqrt{g_x^2 + g_y^2}$$

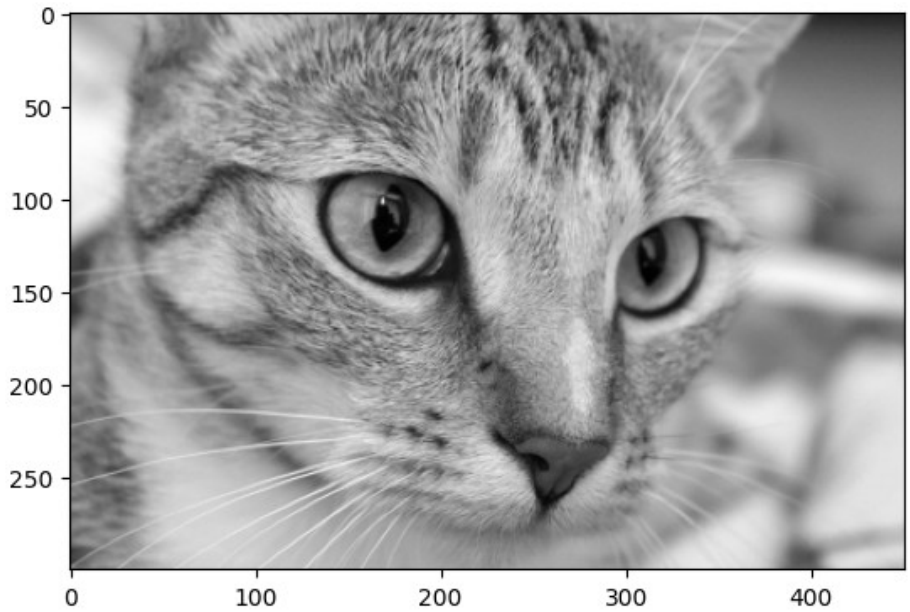
По сути, мы применяем теорему Пифагора.



Найдем контуры на более реалистичном изображении.

```
from skimage import data
from skimage.color import rgb2gray
import cv2

# возьмем фотографию кота из библиотеки skimage
cat_img = data.chelsea()
cat_img_gray = rgb2gray(cat_img)
plt.imshow(cat_img_gray, cmap = plt.cm.gray);
```



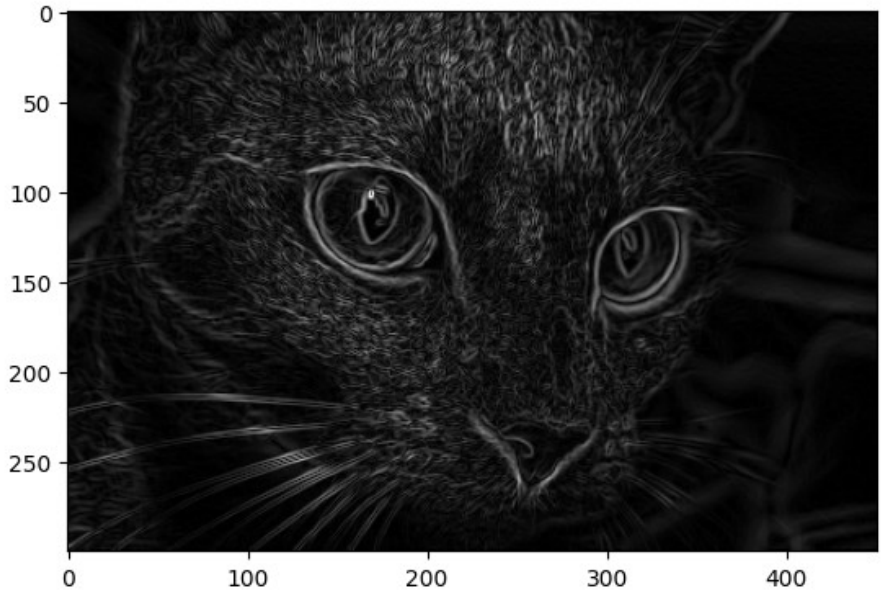
Применим фильтр Собеля из библиотеки cv2.

```
# dx=1, dy=0 указывают на расчет градиента вдоль оси x
```

```
# dx=0, dy=1 - соответственно вдоль оси y
# ksize - размер фильтра (kernel size)
sobel_x = cv2.Sobel(cat_img_gray, cv2.CV_64F, dx=1, dy=0, ksize =
sobel_y = cv2.Sobel(cat_img_gray, cv2.CV_64F, dx=0, dy=1, ksize =

# рассчитаем длину вектора градиента
edge_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)

plt.imshow(edge_magnitude, cmap = plt.cm.gray);
```



Добавим, что фильтр Собеля — далеко не единственный фильтр, способный обнаруживать контуры на изображении.

Выше мы уже использовали **фильтр Прюитт** (Prewitt filter). Кроме этого можно упомянуть **фильтр Шарра** (Scharr filter).

1	0	−1
1	0	−1
1	0	−1

фильтр Прюитт

3	0	−3
10	0	−10
3	0	−3

фильтр Шарра

Примечание. Как и в случае с фильтром Собеля, эти фильтры также

имеют «версию» для обнаружения горизонтальных контуров.

Ключевая идея

Обратите внимание, применив фильтр с определенными (выбранными не случайным образом) значениями, мы можем находить на изображении некоторый объект.

> Можно возразить, что пока что это лишь контур, однако ключевая идея заключается в том, что если подобрать правильные значения фильтра (то есть найти веса в процессе обучения модели), то мы сможем находить любые, не обязательно вертикальные и горизонтальные контуры, а также, собирая эти контуры вместе, распознавать и другие интересующие нас объекты.

Именно эту задачу и призвана решить свёрточная нейронная сеть. Более того, математически это возможно, потому что операция свёртки является дифференцируемой, а значит мы можем найти частные производные функции ошибки относительно весов модели.

Перейдем к рассмотрению архитектуры свёрточной нейронной сети.